Token Up a Notch: Teaching LLMs to Play Mario

Team Members: Ishan Khare, Gabriel Seir, Anthony Zhan
Emails: iskhare@stanford.edu, gseir@stanford.edu, azhan9@stanford.edu

1 Objective

Our project explores the use of open-source large language models (LLMs), such as the Llama (Grattafiori et al., 2024) or Mistral (Jiang et al., 2023) models, as decision-making agents in a reinforcement learning setting. Specifically, we aim to fine-tune an LLM to play `SuperMarioBros` by integrating it into the Model Context Protocol (MCP), a structured interface that facilitates communication between language models and dynamic environments (Anthropic, 2024). Rather than using the LLM as a static planner or single-turn reasoner, MCP allows for closed-loop control where the model observes game state, chooses discrete actions, and receives feedback. We investigate whether fine-tuning an LLM within this loop can result in robust, reward-optimized policies for long-horizon control in arcade-style environments.

2 Related Work

Research on LLM-based agents has surged, particularly in tool-use and web interaction (e.g., ReAct (Yao et al., 2023), Toolformer (Schick et al., 2023), WebArena (Zhou et al., 2023)). While these agents excel in symbolic reasoning and tool invocation, they typically lack continuous feedback integration. Projects like Voyager (Wang et al., 2023) introduce long-horizon decision-making in open-world games, but rely on large proprietary models like GPT-4 (OpenAI et al., 2024) and scripted APIs, abstracting away core RL components.

In reinforcement learning, methods like Dreamer (Hafner et al., 2019a) and PlaNet (Hafner et al., 2019b) demonstrate the success of model-based rollouts in structured environments, while reward alignment via RLHF and self-imitation learning is widely used in preference-driven tasks. However, integrating open-source LLMs directly into a reward-optimizing feedback loop remains underexplored. Our work builds on this gap by embedding LLMs into an RL-compatible control loop using MCP, focusing on whether these models can learn from environment feedback to optimize long-term return.

Some recent blog posts have shown that large language models can play games like Pokémon (Lee, 2025) and Super Mario Bros (Wiggers, 2025) using the Model Context Protocol (MCP). But in these examples, the models are used right out of the box, without any fine-tuning or reinforcement learning. Our work takes this a step further by embedding open-source LLMs into a feedback-driven control loop, asking whether they can actually learn from their environment and improve over time to maximize long-term rewards.

3 Technical Outline

At each timestep, the Super Mario Bros environment serializes the current game state (e.g., Mario's position, velocity, obstacles, enemies) into a structured JSON context and sends it to the LLM. The LLM returns a discrete action (e.g., "right", "jump"), which is executed in the environment. The updated state and received reward are then returned to the LLM via MCP, completing the closed control loop.

We will begin training with behavioral cloning on expert demonstrations collected from human or scripted agents. Once the model has learned a basic policy, we will transition into reinforcement learning using REINFORCE Williams (1992) or PPO (Schulman et al., 2017), directly optimizing the model to maximize cumulative game score. We will investigate techniques like reward shaping, self-imitation, and preference modeling to improve stability and sample efficiency.

We will evaluate the agent across standard RL metrics such as average return, sample efficiency, policy entropy, and action diversity. Additionally, we will track level-completion rate—how often the agent successfully reaches the flag—and generalization to unseen levels.

4 Team Contributions

- **Ishan Khare**: Implement the MCP loop and model-environment interface, co-develop the fine-tuning pipeline, and lead reinforcement learning experiments.
- **Gabriel Seir**: Create expert trajectory datasets, co-implement fine-tuning scripts, and design reward functions and evaluation benchmarks.
- **Anthony Zhan**: Implement reinforcement learning algorithms (REINFORCE/PPO), assist with policy diagnostics, and benchmark model variants.

References

Anthropic. 2024. Introducing the Model Context Protocol. https://www.anthropic.com/news/model-context-protocol

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. 2019a. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603* (2019).

Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. 2019b. Learning Latent Dynamics for Planning from Pixels. arXiv:1811.04551 [cs.LG] https://arxiv.org/abs/1811.04551

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL] https://arxiv.org/abs/2310.06825

Benj Edwards Lee. 2025. *Why Anthropic's Claude still hasn't beaten Pokémon*. https://arstechnica.com/ai/2025/03/why-anthropics-claude-still-hasnt-beaten-pokemon/ Accessed: 2025-04-23.

OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, et al. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL] https://arxiv.org/abs/2303.08774

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems* 36 (2023), 68539–68551.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291* (2023).

Kyle Wiggers. 2025. *People are using Super Mario to benchmark AI now*. https://techcrunch.com/2025/03/03/people-are-using-super-mario-to-benchmark-ai-now/ Accessed: 2025-04-23.

Ronald J Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning* 8 (1992), 229–256.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. 2023. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854* (2023).